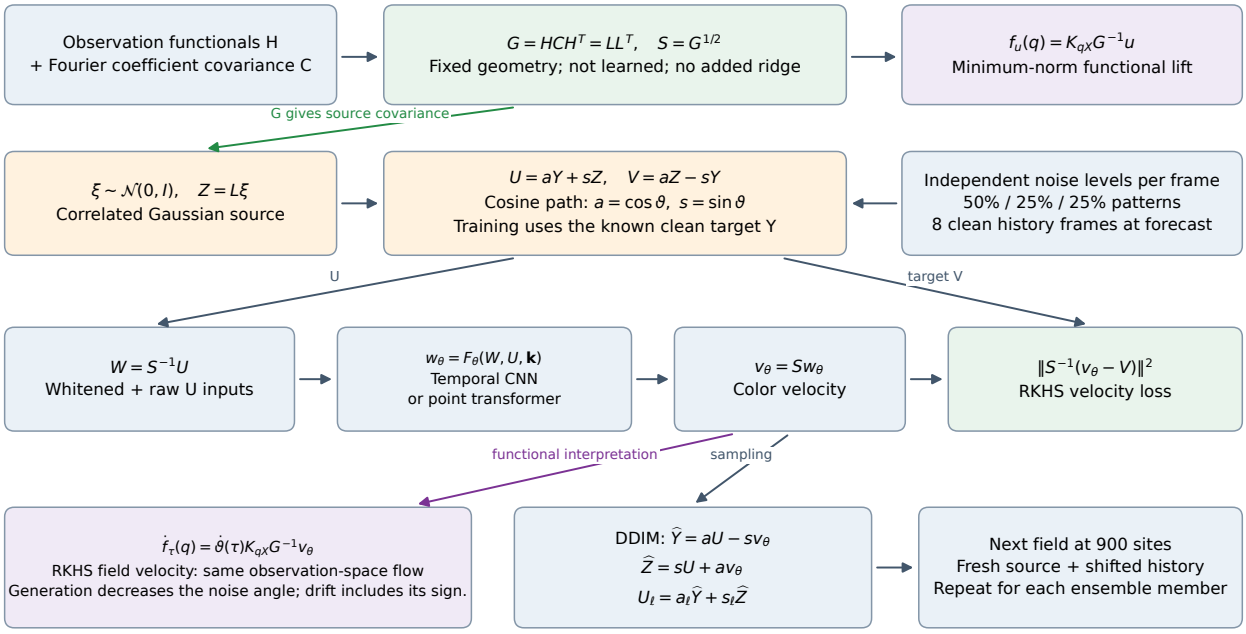
RKHS DFoT: architecture and executed smoke tests

Current DynaBench implementation and explicit StocBench/OceanSSH adaptations

7 October 2026

RKHS flow: lift a learned observation velocity into a field

The same G defines the source law, network coordinates, loss, and functional lift.



Execution advances only the 900 observation values. The fixed linear RKHS lift commutes with those updates.
Off-site evaluation is available as a kernel readout; it is not a hidden dense-grid state inside the trained cloud model.

Evidence boundary. Frozen DynaBench source and run receipts describe the trained method. The new dense adapters were tested with three optimizer updates on real data; their smoke outputs do not establish benchmark accuracy, convergence, or uncertainty calibration.

Current DFoT with RKHS flow

This package documents the implementation behind the completed twelve-case DynaBench DFoT campaign. It includes its exact source snapshots, the existing related-work/theory PDF and LaTeX sources, and a new dense-grid adapter for bounded StocBench and OceanSSH execution tests. These are distinct evidence levels: DynaBench has trained results; the new adapters have smoke tests, not trained benchmark results.

1. What the model evolves

The current DynaBench state is the field **at the 900 supplied observation locations**, not a learned latent vector or an interpolated dense image. A training tensor has shape `[B, T, N, C]`, with batch `B=8`, sequence length `T=9`, observation count `N=900`, and `C` physical channels. Eight frames are history and the ninth is the next physical-time target.

Advection, Kuramoto-Sivashinsky, and predicted Wave displacement have one channel; Burgers and Reaction-Diffusion have two; Gas Dynamics has four. The channel order is the dataset's saved order. Wave predicts only its first field. Each system/layout has separately trained weights; the twelve models do not establish transfer between grid and cloud.

There are two times. Physical time indexes the nine frames and the sixteen-step forecast rollout. Denoising time traverses 1,000 noise levels to generate one next field. A 50-step DDIM sample is one physical transition, not a 50-step physical rollout.

2. Observation-induced geometry and the functional lift

Let `O_X` read a continuous field through the specified observation functionals. Their representers define `G_X = O_X O_X*`, an SPD observation Gram matrix. The minimum-norm lift is

$$S_X y = O_X^* G_X^{-1} y, \qquad O_X S_X y = y.$$

For a finite observation velocity `v`, the lifted field velocity is `S_X v`. At additional query functionals, evaluation is `G_(X',X) G_X^{-1} v`. This is the fixed-geometry lift: it does not require a neural network whose internal state is an entire continuous function. It also does not imply that a separately trained network on a different layout produces the same field.

The induced norm is

$$\|S_X e\|_H^2 = \mathrm{tr}(e^T G_X^{-1} e) = \|G_X^{-1/2} e\|_F^2.$$

The benchmark observation map is specific. `operators.py::Observation.design` reproduces the released periodic thin-plate RBF observation from a 64-by-64 underlying grid using sixteen neighbors, a degree-one polynomial, and zero smoothing. The trigonometric endpoint basis contains all 64-by-64 real modes, including its explicit Nyquist convention. The diagonal coefficient variance is proportional to

$$c_{k_x,k_y} = \left(1 + (k_x^2 + k_y^2)/16\right)^{-3},$$

normalized by the sum over all 4,096 coefficients. If `A` is the observation of the basis already multiplied by the square roots of these variances, `G=A A^T`. This is a finite Fourier prior composed with the released observation operator. The optional `periodic_matern32` helper is **not** the covariance used in the current campaign.

`core.py::RKHSGeometry` constructs a Cholesky factor `L` and the unique symmetric root `S=G^(1/2)`. The latter is obtained from the SVD of `L`, on CPU in FP64, before transferring fixed factors to the GPU. There is no added ridge, eigenvalue clipping, or mode truncation in this construction. Cholesky solves are used for the objective; the symmetric inverse root is used as the network coordinate transform. The symmetric choice avoids treating the arbitrary Cholesky ordering as a learned feature.

The implementation advances observation values. Its minimum-norm lift only covers the observation span. It does not additionally learn or sample an unconstrained full-field null-space component in these DynaBench runs.

3. The source, path, and velocity target

Each frame and physical channel receives an independent Gaussian field source with the same spatial covariance:

$$Z = L\epsilon, \quad \epsilon \sim N(0, I), \qquad U_k = a_k Y + s_k Z, \qquad V_k = a_k Z - s_k Y.$$

The cosine schedule is vendored from DFoT. Index 0 is an added exact-clean state, `(a_0,s_0)=(1,0)`; index 1000 is exactly pure noise, `(a_K,s_K)=(0,1)`. Positive indices 1 through 1000 correspond to the released 0 through 999 schedule. Source, path, target, and loss geometry use FP64; the trainable network uses FP32.

The target is `pred_v`, not `pred_x0`, an ordinary next-field regression target, or the straight-line displacement `Y-Z`. With `a=cos(theta)` and `s=sin(theta)`, `V` is the angular derivative of the noising path. Generation reverses the noise levels. The linear `observed_path` helper remaining in `core.py` is not called by the current DFoT trainer.

The same `G` controls source covariance, whitening, output coloring, and regression. Changing only one of these would implement a different method.

4. Framewise DFoT training

The per-example level-pattern distribution is:

Probability	Levels	Supervision
50%	All nine levels independently uniform in 1 through 1000	All nine frames
25%	Eight exact-clean history frames; future level uniform in 1 through 1000	Future only
25%	Eight exact-clean history frames; future at level 1000	Future only

The objective first averages supervised frames within an example, then examples:

$$\mathcal{L} = \frac{1}{B} \sum_b \frac{\sum_j m_{bj} \operatorname{tr}(E_{bj}^T G^{-1} E_{bj})/(NC)}{\sum_j m_{bj}}, \qquad E = V_\theta - V.$$

Clean history is conditioning information, not a supervised denoising target, in the two forecasting branches. The independent branch teaches joint denoising and gives direct loss gradients to all frames. Uniform level weighting keeps the pure-source endpoint supervised. This is the implementation in `diffusion.py::mixture`, `pair`, and `loss`; the trainer samples the source with `RKHSGeometry.source`.

The theory PDF derives an ELBO whose Gaussian KL terms have the same covariance-weighted regression geometry, but schedule-dependent coefficients. The implemented uniform weighting plus the added forecast patterns is not literally the full exact ELBO, including its reconstruction and prior terms.

5. The neural velocity architectures

Both backbones take raw field values `U` and whitened values `S^-1 U`. They produce a whitened velocity, which is mapped back by `S`:

$$V_\theta(U, k, X) = S\, F_\theta(S^{-1}U,\, U,\, k,\, X).$$

The raw-input projection is initialized at zero and subsequently trained. It provides physical amplitude information while retaining the common source/loss geometry. It is not an identity skip directly from the input to the output. Algebraically, the RKHS quadratic of an output error `S e_w` is the Euclidean norm of `e_w`; this is consistent coordinate matching, not an additional independent regularization penalty.

Regular grid: frame-shared temporal CNN

The 900 values reshape to a 30-by-30 image without interpolation. The same spatial weights operate at all nine frames. A one-by-one embedding for whitened values and a zero-initialized one-by-one raw-value embedding produce hidden features. Per-frame noise levels pass through sixteen sine/cosine frequencies, giving 32 features, followed by a `32 -> 36 -> width` MLP.

Four residual blocks have dilations `[1,2,4,1]`. Each contains channel LayerNorm; framewise noise FiLM shift/scale; a spatial 3-by-3 convolution; a temporal three-tap convolution; another channel normalization and spatial 3-by-3 convolution; and a residual addition. Spatial padding is circular. Temporal edges repeat the first/last frame; time is not circular. The dilation sequence gives the last frame access to all eight history frames. A normalized spatial head produces all nine velocity frames, then `S` colors the output.

Grid channels	Width	Depth	Parameters	Cases
1	40	4	151,869	Advection, KS, Wave
2	41	4	159,860	Burgers, RD
4	42	4	168,520	Gas Dynamics

Some historical config descriptions say ”42 features” generically. The instantiated widths above come from the actual per-run configs and parameter counts. `heads`, `patch_size`, and `mlp_ratio` fields retained from earlier grid architecture experiments do not turn this model into a transformer: `GridDFoT` does not use them for attention or patchification.

Cloud: point transformer

Nine frames and their channels are concatenated per location. Separate raw and whitened linear embeddings map `9*C` values into 128 features. The level embedding concatenates all nine per-frame 32-component sinusoidal embeddings, followed by a two-layer MLP. Six vector-attention blocks operate on a periodic sixteen-neighbor graph, using wrapped two-dimensional displacements.

Each block includes noise FiLM, query/key/value projections, a learned relative-position embedding, channelwise neighbor attention, a global mean feature, and a feed-forward residual. The output head returns `9*C` values per point, which are reshaped and colored by `S`. Temporal mixing is through the concatenated frame features and shared level embedding, not a causal temporal attention mask. No dense image is constructed by this cloud backbone.

Cloud channels	Width	Depth	Parameters
1	128	6	1,445,897
2	128	6	1,449,362
4	128	6	1,456,292

6. DDIM forecasting and physical rollout

Inference concatenates eight clean history frames with one correlated source field. The history levels stay exactly zero. For the current future level `k`, recover

$$\widehat{Y} = a_k U - s_k V_\theta, \qquad \widehat{Z} = s_k U + a_k V_\theta, \qquad U_\ell = a_\ell \widehat{Y} + s_\ell \widehat{Z}.$$

The deterministic DDIM solver uses fifty rounded, uniformly spaced level transitions from 1000 to 0. The physical state passed to the network is FP32; the schedule arithmetic is FP64 before assignment back to that state. No extra solver noise is injected. Different source draws still generate an ensemble. The sampler checks that the clean history is bitwise unchanged.

For the sixteen-step physical rollout, shift the history and append the predicted next frame, then use a fresh source. Each of the four ensemble members keeps its own generated history; the next transition is not conditioned on the ensemble mean. The reported leadwise MSE averages the four members first, then measures error against truth over the 32 fixed validation windows, spatial points, and physical channels.

7. Optimization and evidence

Current DynaBench training uses AdamW, betas `(0.9,0.99)`, weight decay `0.001`, gradient clipping at 1, 1,000-update warmup, and ramped EMA with maximum decay `0.999`. Geometry and diffusion computations remain FP64 around an FP32 network. The deterministic data schedule visits all 5,000 training trajectories, sampling eight windows per visit. Configs and generator states are included in checkpoints. Resume tests preserve model, EMA, optimizer and random streams.

Budgets and learning-rate histories differ between systems, including continuation stages. Exact run/source mappings are in `records/current_run_sources.json`, current stage configs and completion/review receipts in each corresponding records directory, and final results in `records/campaign_status.json`. Some stage config files record the stage-start LR; the latest review/checkpoint identifies a later reduced LR. A completed stage is not a convergence certificate.

Final lead-16 DFoT scores are numerically below the best recorded paper target in nine of twelve cases. Grid Advection, grid RD and cloud RD miss those targets. These are descriptive published-score comparisons: the local 32-window, four-draw validation protocol is not the paper's recovered protocol. Same-backbone deterministic controls have lower point-forecast MSE on all six grid cases and cloud Advection; DFoT has lower MSE on the other five cloud cases. The controls belong in the supplement, as agreed, and main claims must remain consistent with them. Probabilistic value needs calibration/proper-score evidence in addition to point MSE.

8. What is and is not physically enforced

The periodic geometry and wrapped grid/cloud interactions encode the DynaBench domain topology. The source covariance and RKHS metric encode a finite spectral smoothness preference. History is held exactly fixed during forecasting. These are concrete model properties.

The model does not impose a PDE residual, mass or energy conservation, positivity, incompressibility, a maximum principle, or long-time stability by construction. The theory's transport and rollout bounds have explicit regularity, sensitivity, and sometimes contraction assumptions. Those assumptions are not established for every benchmark merely by training this network. A sampled ensemble is not automatically calibrated, and finite-span consistency is not a proof of arbitrary-resolution generalization.

9. New StocBench/OceanSSH adaptation

`code/dense_adapter/network.py` imports the exact frozen current grid CNN class and blocks. It generalizes frame/grid sizes, adds external condition embeddings, and replaces dense Gram operations with full-rank spectral transforms. It does not reuse the older `functional_dense` patch transformer, white source, `pred_x0`, or min-SNR objective. The old real-data loader and native evaluation wrapper are reused as data/integration components only.

Property	Current DynaBench grid	New StocBench smoke	New OceanSSH smoke
Resolution	30 x 30, observed from 64 x 64	64 x 64 dense	200 x 200 I-III; 240 x 240 IV
Sequence	Eight history + one future	One history + one future	Nine jointly reconstructed days
CNN	Four residual blocks	Same block code, width 42	Same block code, width 42
Spatial boundary	Circular	Circular	Replicated regional edges
Gaussian geometry	Observation-induced dense Gram	Full periodic Fourier covariance	Full nonperiodic cosine covariance
Diffusion	Correlated `pred_v`, RKHS quadratic	Same construction	Same construction, reconstruction patterns
External conditions	Clean history in state	Current forcing only for deterministic task	SSH/SST values and availability masks
Weights in this package	Frozen source, trained-run receipts	Fresh smoke-trained weights	Fresh smoke-trained weights

For dense grids, write `G = Q diag(lambda) Q^T`, where the two-dimensional transform is the tensor product

of orthonormal one-dimensional bases. Apply `G^(1/2)`, `G^(-1/2)` and `G^-1` by transforms and modal multiplication. All modes remain present with positive variance; no full 40,000-by-40,000 or 57,600-by-57,600 matrix is formed. At 64 periodic sites, the source covariance matches the current Fourier endpoint kernel for direct dense observations, including the Nyquist normalization. It is not the 900-reading RBF Gram, since the observations differ.

Ocean uses a cosine basis on normalized pixel centers and the same positive spectral decay formula. This is a declared new regional covariance, not a recovered OceanSSH physical prior or the original periodic observation Gram. Cosine modes have a reflecting extension at patch edges; the CNN repeats edge values rather than wrapping across opposite edges. This boundary treatment is an architectural choice, not an imposed ocean boundary law. Spatial padding is implemented by concatenation to retain deterministic CUDA backward on the installed PyTorch version.

10. StocBench data and conditioning

Both stochastic and deterministic tasks use their actual released 64-by-64 trajectory arrays, all three training shards (seeds 42, 43, 44), with 1,500 trajectories and 298,500 eligible adjacent pairs available. The smoke selects the first training pair. Native field normalization is identity.

The stochastic input contains only current vorticity; forcing is hidden. The deterministic input adds only the forcing for the current physical interval. The adapter broadcasts that condition across the two-frame network; it does not read the next interval's forcing as an input. The original 50/25/25 level mixture is retained with one clean history frame instead of eight. Native `sample(cond, sampling_schedule=('u', steps))` returns the next vorticity image. The smoke verifies exact history preservation, finite autoregressive output, and conditioning dependence. Native callback tests additionally exercise the released metric integration with one condition, four members, and two solver/rollout steps.

The released workflow has no independent validation split here; the smoke's second loss is explicitly a training probe. Three updates and the bounded native metric run are execution evidence only. They do not assess converged stochastic accuracy, calibration, energy distance reproduction, long-horizon behavior, or deterministic ensemble collapse.

11. OceanSSH I-IV data and conditioning

Experiment	Dense training source	Observation conditions	Smoke grid
I	NATL60 SSH	NADIR SSH values + mask	200 x 200
II	NATL60 SSH	NADIR + SWOT SSH values + mask	200 x 200
III	NATL60 SSH	NADIR SSH and SST, each with mask	200 x 200
IV	NATL20 SSH with simulated sampling	Real NADIR at inference; CryoSat-2 excluded	240 x 240; exported inner 200 x 200

NATL60 uses Jan 2-Aug 31, 2013 for training and September 2013 for validation. Its 2012 scoring period is separate. NATL20 uses Oct 1-Dec 14, 2012 and Feb 24-Sep 29, 2013 for training, and Dec 15-Feb 23 for validation. Windows cannot cross a split boundary or a missing date. Train-only finite-value statistics normalize SSH and, for III, SST. IV inference preserves the established transfer convention of using observed real SSH statistics without held-out CryoSat-2 measurements.

The condition vector is `[SSH value, SSH availability, SST value, SST availability]`. Missing values are zero-filled; the network additionally multiplies value payloads by masks, so arbitrary payloads at missing sites cannot leak into inference. III's SST channels are tested separately. `reconstruct` accepts only conditions and optional source, not a dense truth tensor; the inference loader is guarded against requesting the NATL60 dense target file. The sparse observations supplied by the release are on a grid; the adapter does not pretend they are complete clean history frames.

All nine SSH frames are unknown generated states. The adapted training mixture is 50% independent positive levels, 25% one shared random positive level across all frames, and 25% all frames pure noise. All available target frames are supervised. This reconstruction mixture is an explicit adaptation: it is not the DynaBench

clean-history forecast template. Sampling starts all frames from the correlated source and denoises them synchronously while conditioning on observations. No exact measurement projection or Bayesian posterior guarantee is claimed.

When training targets have missing cells, the implemented loss sets residuals at those cells to zero before applying the full precision quadratic and normalizes by the valid count. This is a masked-residual quadratic, **not** the inverse of the observed sub-covariance and not an exact marginal Gaussian likelihood. Mean-filled missing target values can still enter the noisy state as an adaptation choice. A full scientific campaign should review this treatment or use valid-domain/marginal-covariance modeling. The smoke reports finite target coverage explicitly.

The four conditional models are initialized fresh for three-update execution checks. No dense-prior pretraining, full-period reconstruction, OceanBench score, or reactivation of the stopped historical 4DVarNet runs is part of these tests. The nine-day reconstruction window also differs from the historical fifteen-day 4DVarNet setup. The exported NetCDF is one bounded window, not a complete benchmark product.

12. Evidence and code map

Question	Active source in each frozen DynaBench directory	New dense adapter source
Covariance and lift	`endpoint.py`, `operators.py`, `core.py::RKHSGeometry`	`network.py::SpectralGeometry`
Frame patterns and velocity target	`diffusion.py::mixture`, `pair`	`network.py::mixture`, `pair`
Loss and supervised reduction	`diffusion.py::loss`	`network.py::metric`, `loss`
Grid velocity network	`model.py::GridDFoT`, `Block`	Imported unchanged blocks; `DenseFGD.forward`
Cloud velocity network	`model.py::CloudDFoT`, `point_backbone.py`	Not used for these dense datasets
Forecast sampler	`diffusion.py::sample`	`network.py::denoise`, `sample`
Reconstruction sampler	Not the DynaBench task	`network.py::reconstruct`
Data/splits	`dataset.py`, `data.py`, `config.py`	`stoc_data.py`, `ocean_data.py`, `configs/`
Training/resume	`train.py`, campaign controller context	`smoke.py::run`, snapshot/replay checks
Saved-output scores	`evaluate.py`, `verified.json` receipts	`smoke.py`, native `evaluate_stoc.py`

The smoke first compares transform-based geometry against explicit small dense SPD inverses, including loss gradients. It checks the periodic-64 covariance against the current endpoint basis and compares the generalized CNN against the frozen original at nine frames and 30-by-30 resolution with identical weights. Each real-data case forces all three objective branches, checks finite nonzero updates, replays one interrupted update including EMA/optimizer/RNG, strictly reloads its checkpoint, and samples at fifty denoising steps. Ocean adds mask/SST/observation guards and a NetCDF roundtrip. See `SMOKE_RESULTS.md` and the raw machine-readable receipts for actual outcomes and limitations.

The theory PDF is copied unchanged at the package root. `theory/` contains its editable sources and original evidence notes. This architecture document explains implementation details and explicitly marks the new benchmark adaptations; it does not silently extend the original theory claims to those adapters.

Visual guide to the RKHS flow architecture

Read `rkhs_flow_visual_guide.pdf` alongside the revised `figures/architecture_diagram.pdf`. The diagram now shows how the same covariance appears in four places: the correlated source, network whitening/coloring, the training metric, and the minimum-norm field lift. The ten worked pages show what those operations actually do.

All data examples use the first saved cloud-Advection validation record: simulation 953, start 17, field 0, and the 90k EMA checkpoint. Selection did not use its forecast error. Spatial panels show 900 discrete points, without image interpolation or triangulation. The one off-site panel evaluates the finite RKHS field at 900 explicitly new query points; those values are a mathematical readout of the lift, not additional ground-truth measurements.

Symbol-to-example map

Item	Meaning in the implementation	Visual example
X	Native locations of the 900 released observation functionals	All native spatial panels, starting with page 1
Y	Clean next-field readings, used as the supervised training endpoint	Page 1 observed field; page 3 clean endpoint
H and C	Released observation of the finite Fourier basis; diagonal prior coefficient covariance	Page 9 observed modes, prior variance, and observation matrix; page 1 Gram
G = H C H^T	Covariance and RKHS Gram matrix at the observation sites	Page 1 kernel column and Gram submatrix
L, S	Cholesky root and symmetric positive root of the same G	Page 2 source construction; page 4 coordinate transforms
xi	Independent Gaussian coefficients used for the source	Page 2 first spatial panel
Z = L xi	Spatially correlated source at native sites	Page 2 middle spatial panel
S^-1 Z	A white Gaussian realization in symmetric coordinates	Page 2 right spatial panel
a_k, s_k, vartheta	Exact discrete cosine-schedule coefficients and their angular parameter	Page 3 coefficient plots and quarter-circle plot
U = a Y + s Z	Noisy state on the supervised corruption path	Page 3 first row, five levels
V = a Z - s Y	Angular velocity regression target	Page 3 second row, five levels
k and m	Per-frame noise levels and supervised-frame mask	Page 6 two heatmaps with explicit values
W = S^-1 U	Whitened network input, used alongside raw U	Page 4 first row
w_theta	Network output in whitened velocity coordinates	Page 4 bottom left
v_theta = S w_theta	Predicted velocity in observation-value coordinates	Page 4 bottom middle, beside the target
e and S^-1 e	Velocity error before/after whitening	Page 5 bottom row
e^T G^-1 e	RKHS regression penalty	Page 5 matched-point-MSE counterexample and actual frame loss
S_X u	Minimum-norm field whose released observations equal u	Page 1 re-observation identity and off-site field readout
S_X v_theta	Functional velocity induced by the learned observation velocity	Page 10 lifted field path, target field velocity, and learned field velocity
Y_hat, Z_hat	DDIM clean/source estimates recovered from U and v_theta	Page 7 sampler snapshots and the diagram's DDIM equations

Item	Meaning in the implementation	Visual example
Conditional ensemble	Multiple source draws under the same conditioning information	Page 8 individual members, mean, and spread
Physical rollout	Repeated next-field generation, shifting each member's own history	Page 8 lead-16 maps and leadwise diagnostic curve

How the RKHS flow enters the computation

The model advances a vector of observation values. The fixed lift assigns that vector a field:

$$f_u(q) = K_{qX} G^{-1} u.$$

Because the lift is linear and fixed for a geometry, lifting a velocity or differentiating the lifted field gives the same result:

$$\dot{f}(q) = K_{qX} G^{-1} \dot{u}.$$

This is the concrete connection between the finite neural velocity and the RKHS field flow. The current cloud model does not need to materialize a dense field during every network call. The same G also specifies the source covariance, the symmetric network coordinate transforms, and the velocity error norm. The revised diagram shows these uses separately instead of hiding them inside one generic backbone box.

For the release observation operator, H consists of local RBF readouts of an underlying finite Fourier field. Consequently, the exact identity is `O_X f_u = u`; it is not generally the assertion that direct point evaluations `f_u(X_i)` equal those RBF readouts. The off-site demonstration uses direct basis evaluation, with the appropriate cross-kernel, while the re-observation check uses the actual released H.

Why the cosine path needs two rows

The first row on page 3 shows U at five levels; the second shows the corresponding velocity target V. If `a = cos(vartheta)` and `s = sin(vartheta)`, then `dU/dvartheta = V`. Noising increases the noise angle vartheta. Generation decreases vartheta, so the generation-time drift includes the factor `dvartheta/dtau` and its sign. It would be incorrect to describe generation as simply advancing in the positive V direction.

The coefficients are taken from the actual discrete diffusion schedule. The visualization does not replace that schedule with an assumed angle linear in the integer noise level. The quarter circle distinguishes the cosine construction, `a^2+s^2=1`, from a straight-line blend, `a+s=1`.

The training construction knows both Y and Z. A real sampler knows history and Z but does not receive the future Y. Page 7 therefore shows a separate trajectory: the current EMA network's own fifty-step DDIM output. It is not labeled as an exact traversal of the supervised path on page 3.

Why the metric example uses equal point MSE

Page 5 constructs two errors from eigenvectors of the actual G. Both have point MSE exactly one. Their RKHS penalties are the inverse eigenvalues and can be very different. This isolates what the geometry changes without conflating it with a change in point-error magnitude. The bottom row then shows an actual trained velocity error and its whitened counterpart. These constructed errors are explanatory examples, not benchmark predictions or results from additional training.

Numerical checks behind the pictures

The native FP64 coordinates were reread through the frozen current data loader. The saved source reconstructs from its Cholesky-white coefficients to maximum absolute error 5.33e-15. Re-observing the minimum-norm lift recovers Y within 6.89e-14; along the five illustrated path levels the maximum error is 1.33e-12. The actual k=500 frame loss is 0.000539394920609, whether computed by the precision solve or by squared whitened error.

The learned fifty-step replay uses all four saved sources and the original clean history. It matches the official frozen sampler exactly and the saved forecast within 4.77e-7; the small saved-output difference is within the 2e-6 replay tolerance. All eight history frames are preserved exactly. The source checkpoint and forecast hashes are recorded in `figures/visual_guide/provenance.json`.

Each spatial figure is an example, not a new aggregate performance claim. The four-member spread illustrates conditional variability, not calibration. The covariance spectrum and Gram image describe matrices rather than interpolated spatial fields. Shared spatial color scales are used when the compared values have the same units; whitening panels carry their own scales because coordinate transforms change magnitudes.

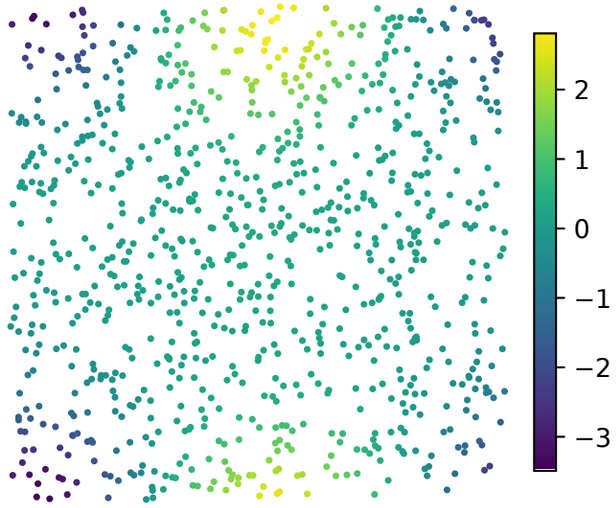
Rebuild

`code/build_visual_guide.py` builds the revised diagram, ten individual PDF/PNG figures and the combined visual-guide PDF. It reuses the compact `figures/visual_guide/worked_example.npz` cache. Recreating that cache requires the existing GPU environment, local benchmark files and the recorded 90k checkpoint; doing so performs only bounded inference, not training. `code/build_documentation.py` includes this guide in the architecture companion when the visual files are available.

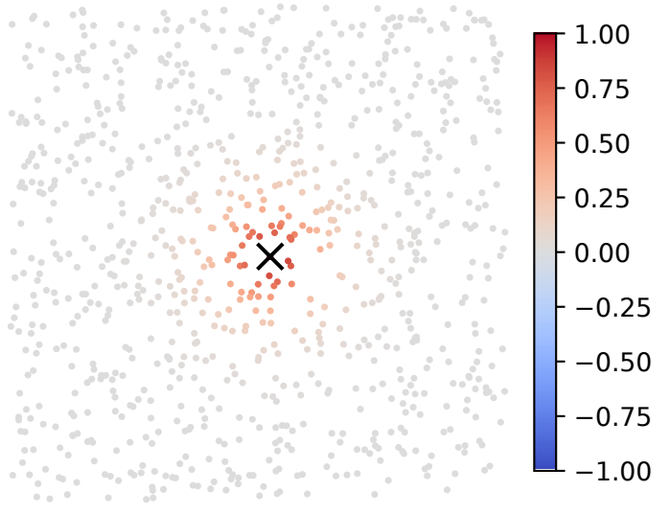
1. The RKHS lift turns readings into a field

Actual cloud Advection example: validation simulation 953, start 17, field 0. Native spatial panels use 900 points.

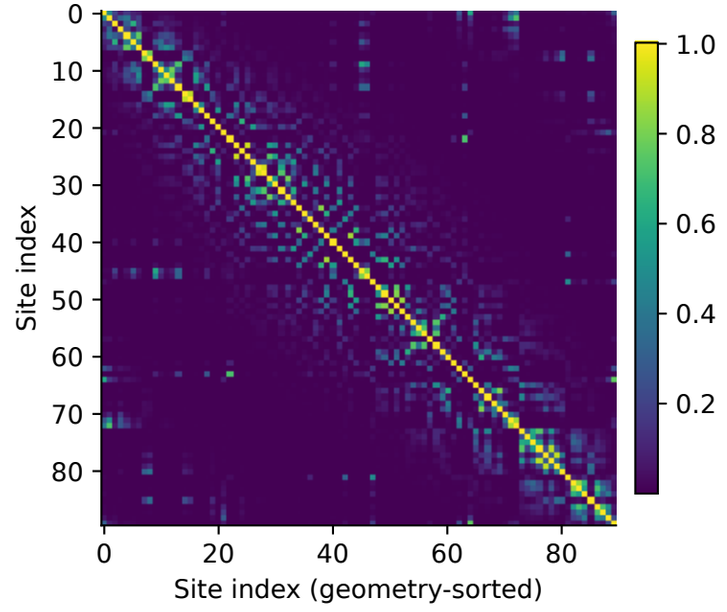
Observed next field $Y \in \mathbb{R}^{900}$



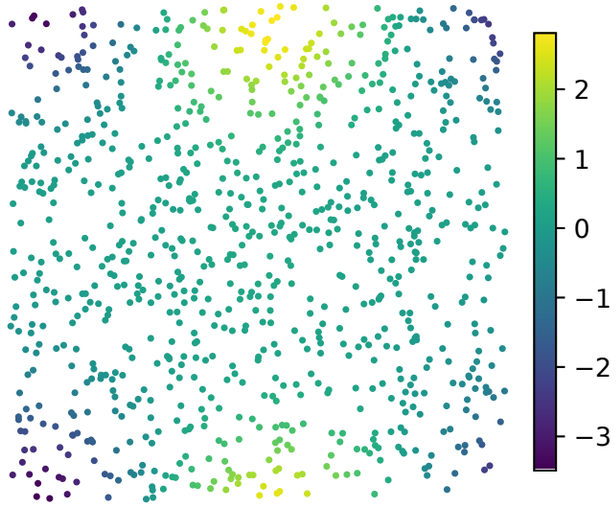
Normalized observation-kernel column



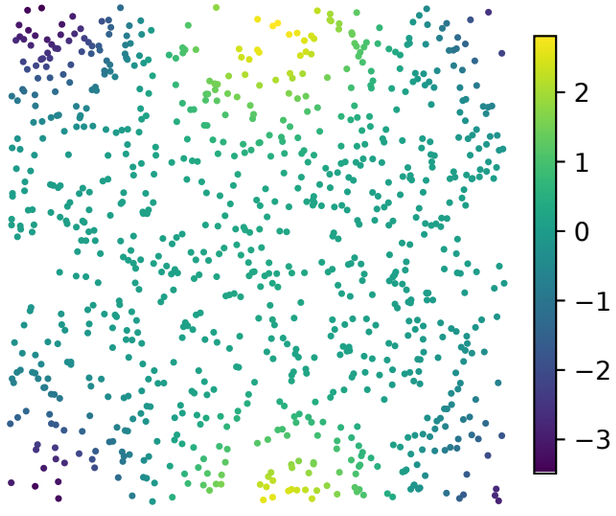
Gram matrix: 90-site submatrix



Re-observe the lift: $O_X f_Y = Y$



$f_Y(q)$ at 900 new query points



$$f_u(q) = K_{qX} G^{-1} u$$

$$\dot{f}(q) = K_{qX} G^{-1} \dot{u}$$

Re-observation error: 6.88e-14
Path-lift error: 1.32e-12

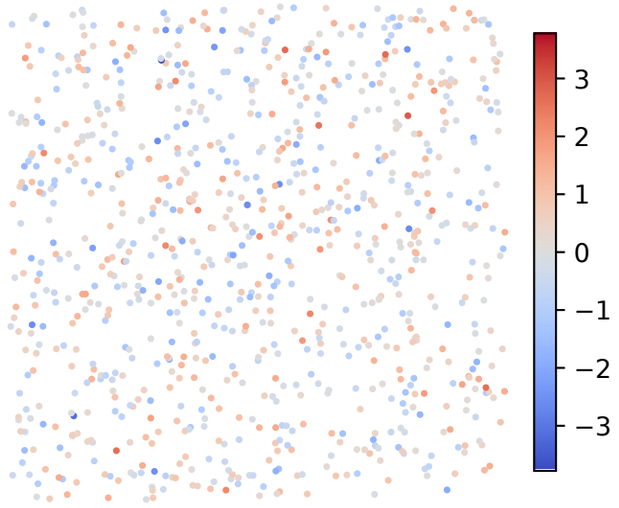
A field velocity is defined by the same linear lift as the field itself.

The native observations are released RBF functionals. New q sites use true point-evaluation functionals of the finite basis.
The off-site panel is an analytic RKHS readout, not ground truth or an interpolated display of the 900 measurements.

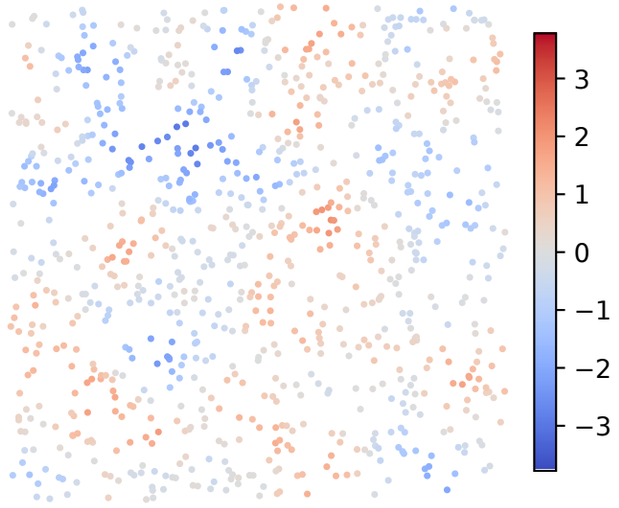
2. Correlated source: random values from one field law

One saved source draw: $Z = L\xi$, $LL^T = G$. Whitening with S^{-1} removes the covariance, not the randomness.

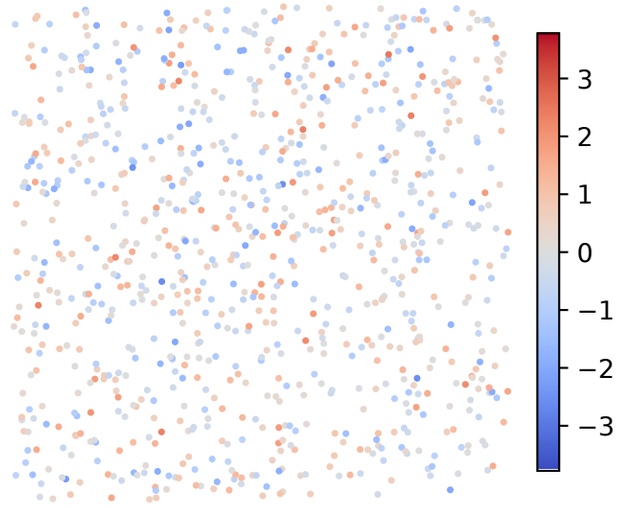
Independent coefficients $\xi = L^{-1}Z$



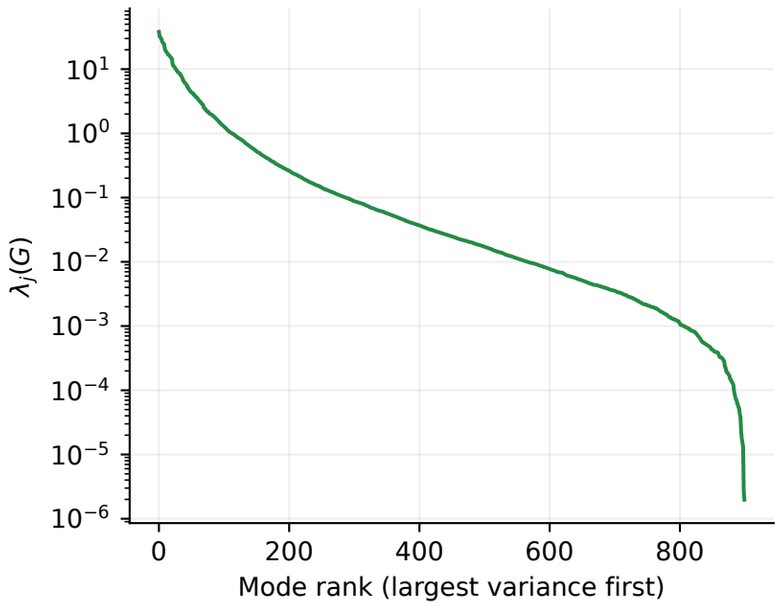
Correlated source $Z = L\xi$



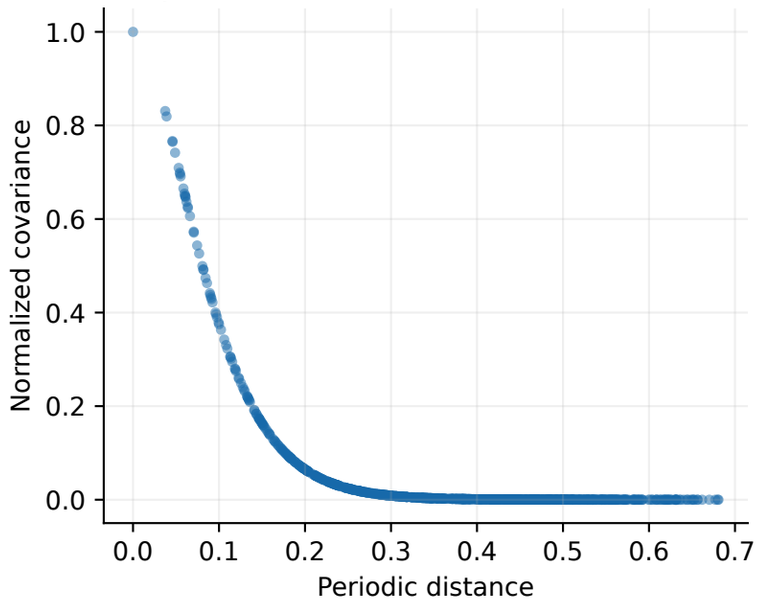
Symmetric whitening $S^{-1}Z$



Covariance eigenvalues



Spatial correlation about marked site



$\mathrm{Cov}(Z) = G$

$\mathrm{Cov}(S^{-1}Z) = I$

$L \neq S$ in general.

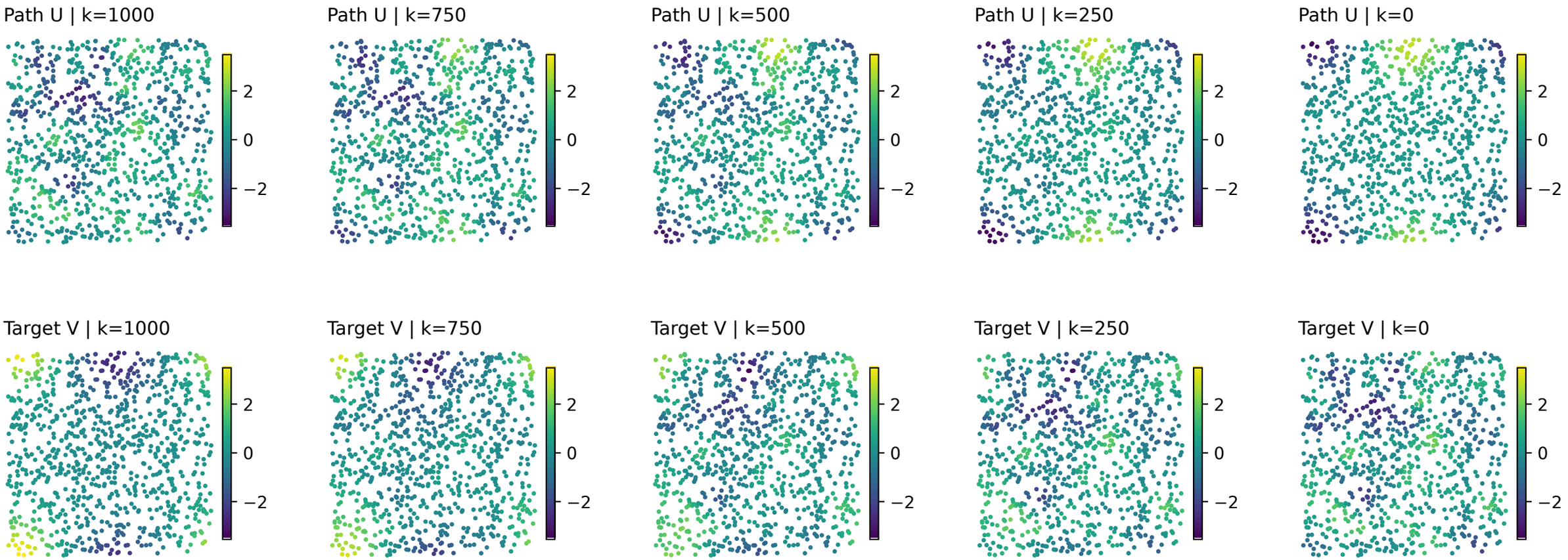
Both roots generate the same Gaussian law.
The first and third panels need not
be the same white-noise realization.

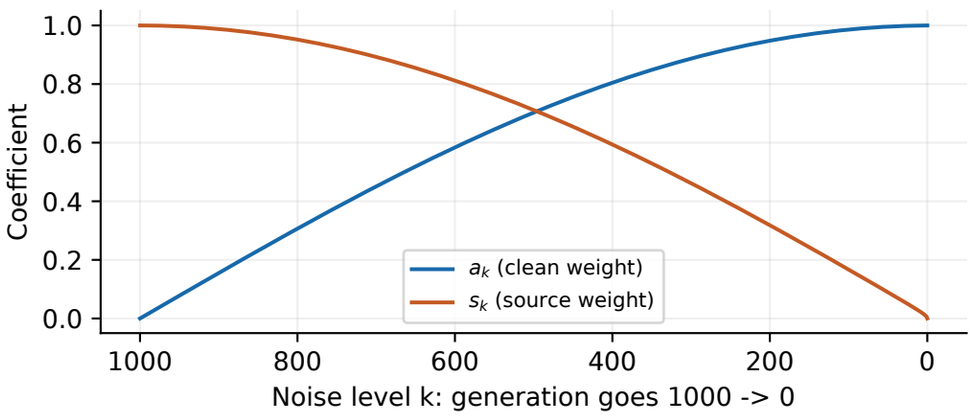
Source reconstruction error: 5.33e-15

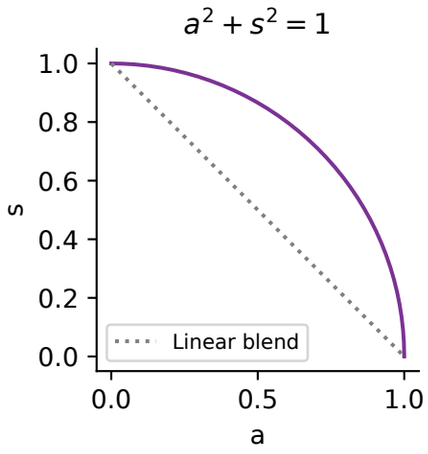
Spatially correlated Gaussian noise provides coherent random structure at the observation sites.
It is not a deterministic initial field, not per-point independent noise, and not a calibration guarantee.

3. The cosine path and its velocity target

A known clean target Y and one fixed source Z define a training pair. Shown left-to-right in generation direction.







$U = aY + sZ$
$V = aZ - sY = \partial U/\partial\vartheta$
$a = \cos\vartheta,\ s = \sin\vartheta$

Generation decreases ϑ: the drift is

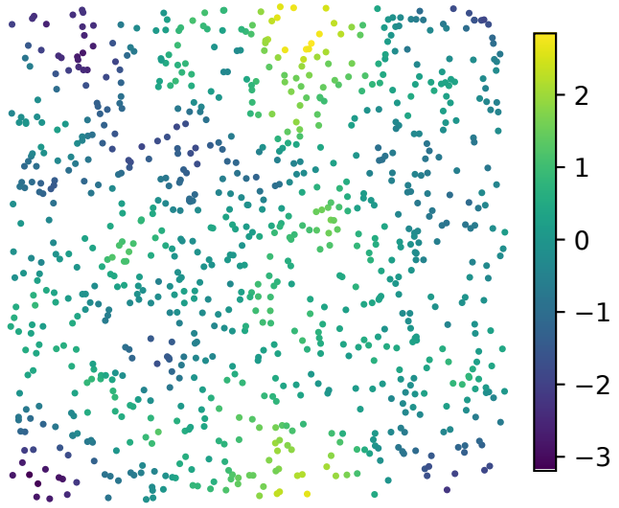
$\dot{U} = \dot{\vartheta}\, V$, with its negative sign.

This is the supervised corruption path, not the trajectory a trained model must follow at inference.
The current discrete cosine schedule sets vartheta(k); the angle is not assumed linear in k. At k=1000, U=Z and V=-Y.

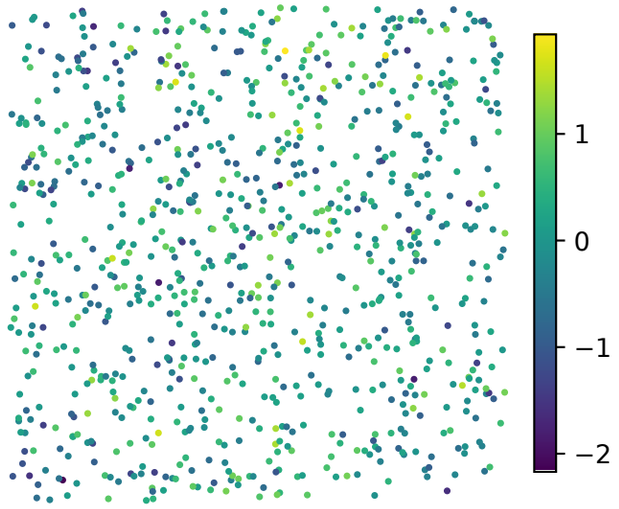
4. What the velocity network sees and returns

At k=500 with eight clean history frames. Each panel has its own numeric color scale; units change under whitening.

Raw noisy values U



Whitened values $W = S^{-1}U$

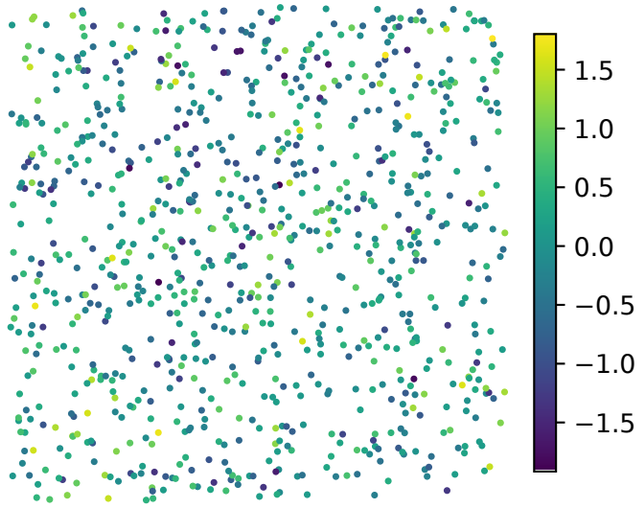


Current cloud network

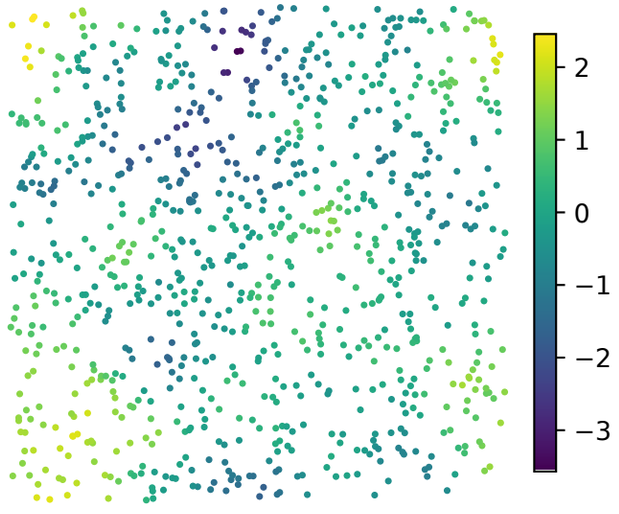
9 frames x channels per point
Raw + whitened linear projections
Per-frame noise embedding
6 point-transformer blocks
128 features, 16 periodic neighbors
9 velocity frames out

$w_\theta = F_\theta(W, U, \mathbf{k})$

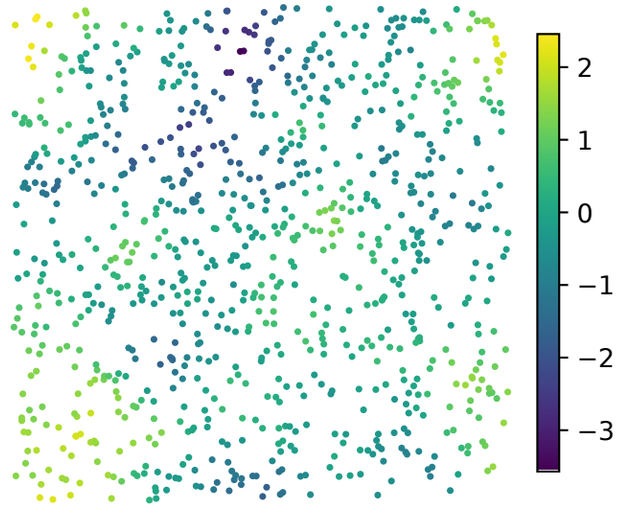
Whitened velocity w_θ



Colored prediction $v_\theta = Sw_\theta$



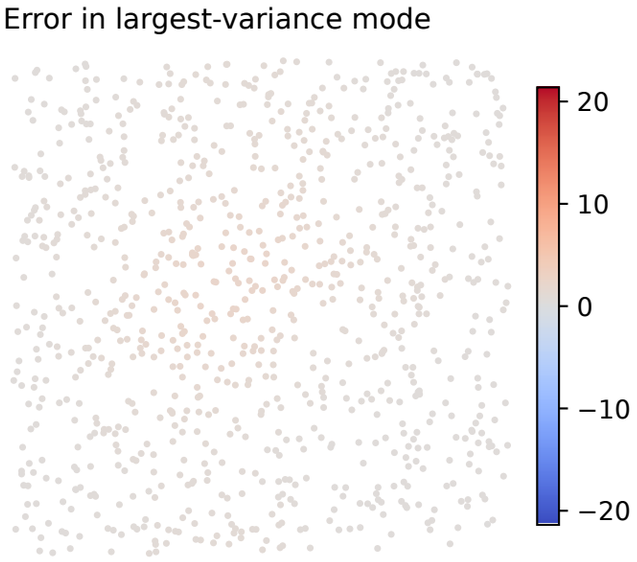
Training target $V = aZ - sY$

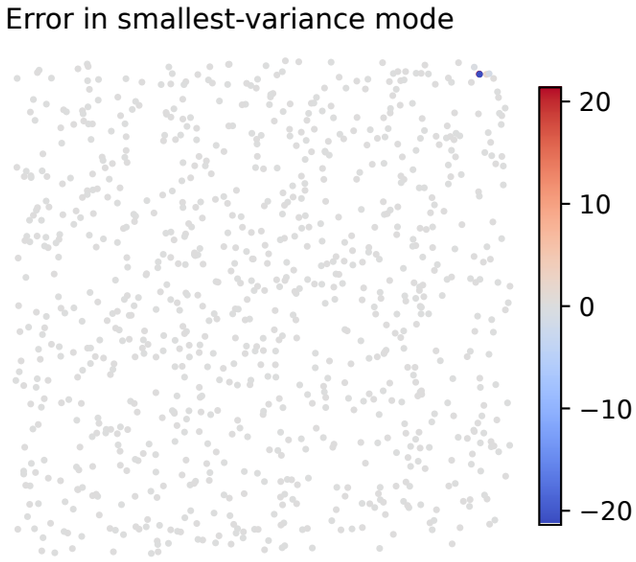


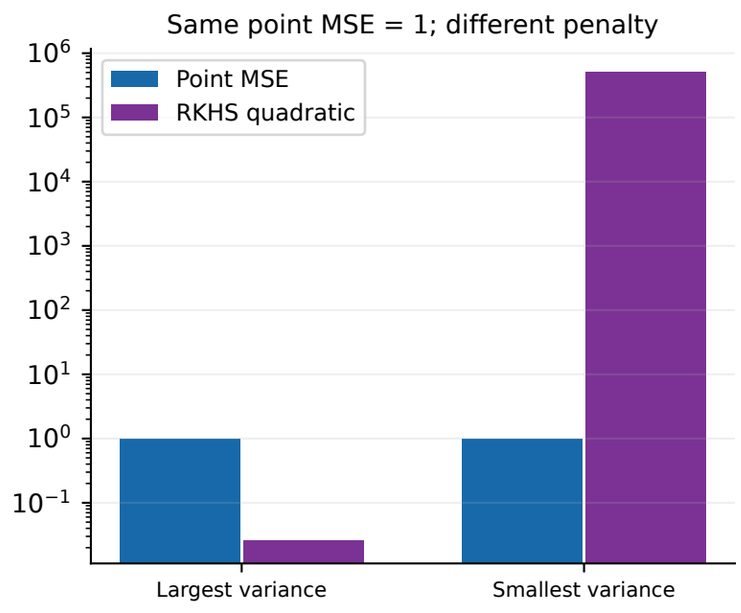
The RKHS geometry surrounds the learned backbone: inverse root at input, matching root at output.
The raw-value branch is also learned. No dense interpolated image is constructed by the cloud point transformer.

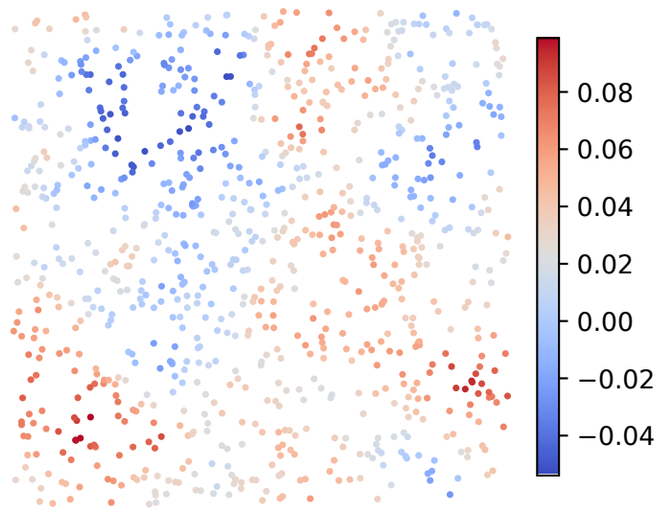
5. The RKHS loss distinguishes errors with the same point MSE

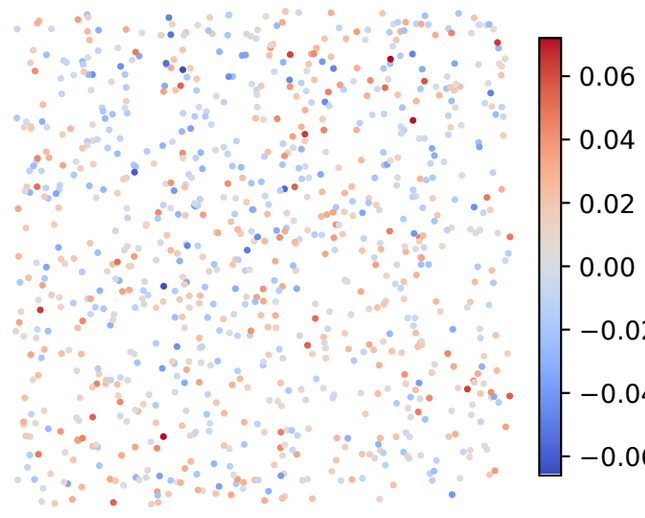
Top row: constructed covariance eigenmode errors. Bottom row: actual velocity error at k=500 in this worked example.











$e^T G^{-1} e = \|S^{-1} e\|^2$

$G q_j = \lambda_j q_j$

$\|\sqrt{N}\, q_j\|^2 / N = 1$

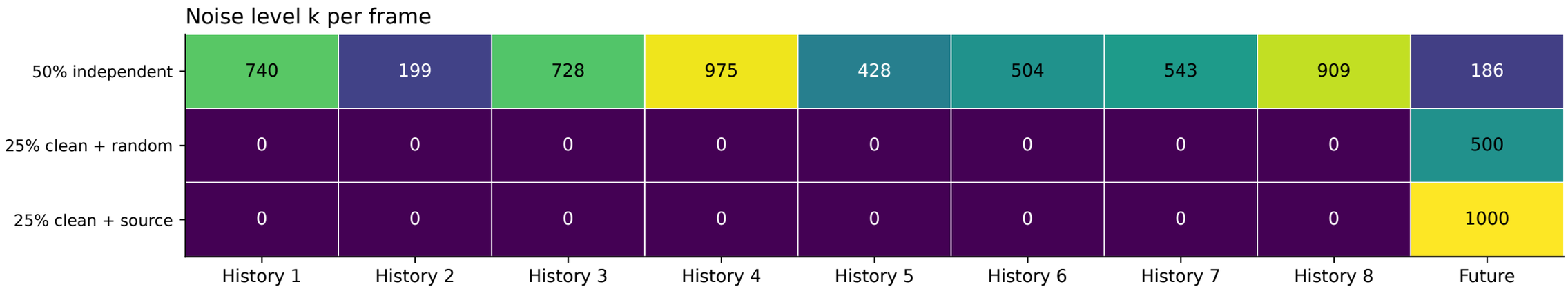
$\|\sqrt{N}\, q_j\|_G^2 / N = 1/\lambda_j$

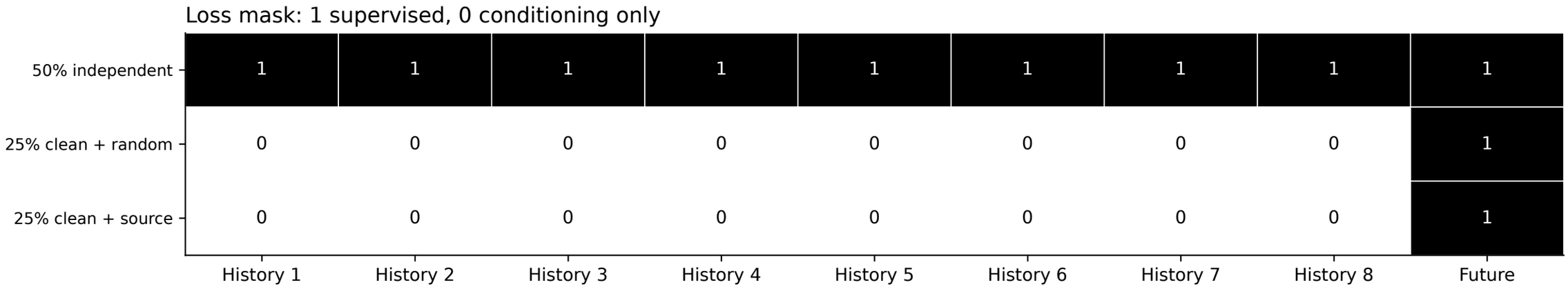
Actual frame loss via solve: 0.000539395
Via whitening: 0.000539395

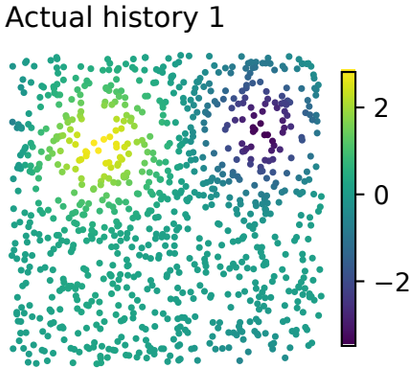
Low-prior-variance directions receive larger precision weights. The full objective also averages supervised frames and batch examples.
This is a covariance-weighted velocity regression loss, not a PDE residual or a guarantee that forecasts conserve physical quantities.

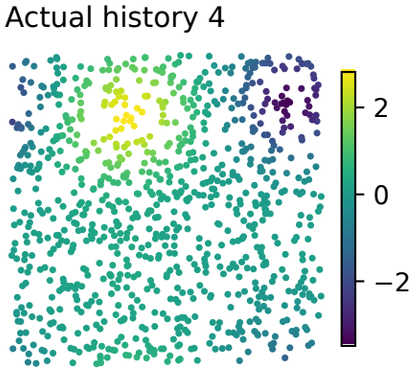
6. DFoT trains with different framewise noise patterns

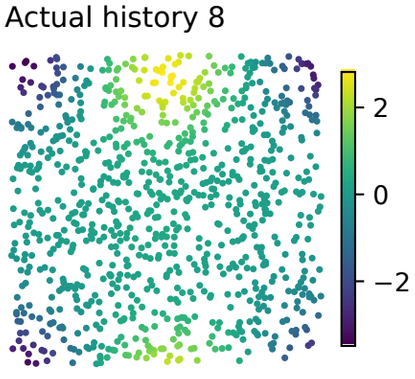
One illustrative level pattern per branch. These are valid examples of the specified mixture, not a new training run.

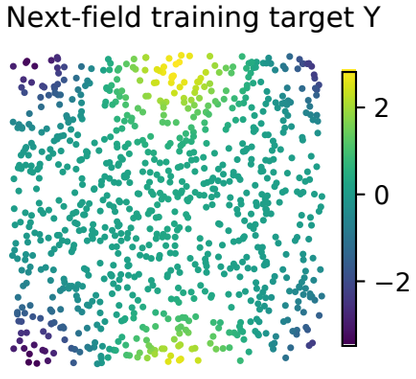








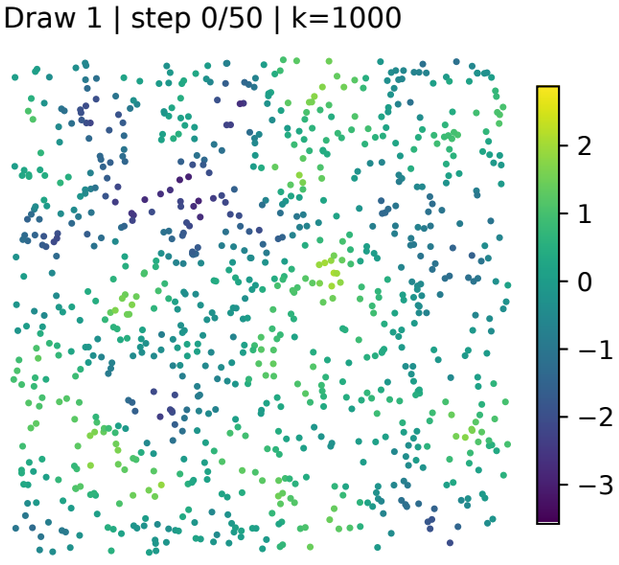


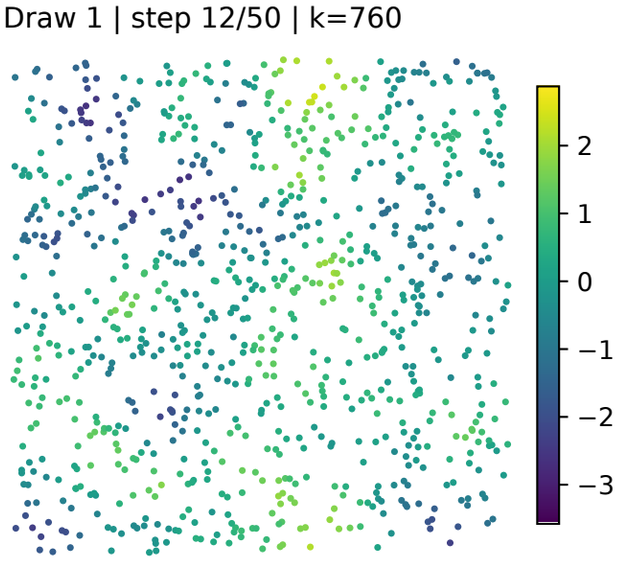


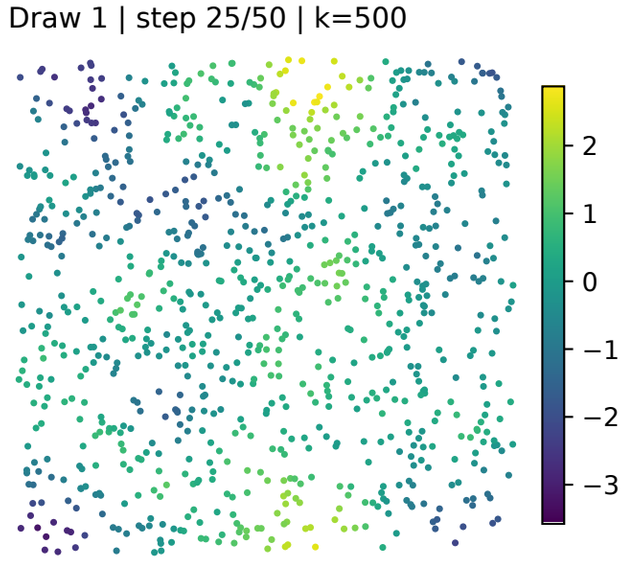
During forecasting, history is held exactly clean; only the future frame is denoised.
The independent branch trains joint denoising. Clean-history branches supervise the future only; no clean-history loss is added.

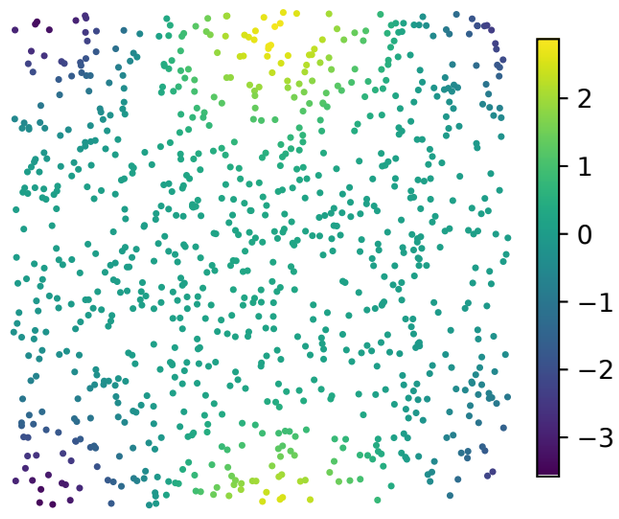
7. Learned DDIM sampling is a different trajectory

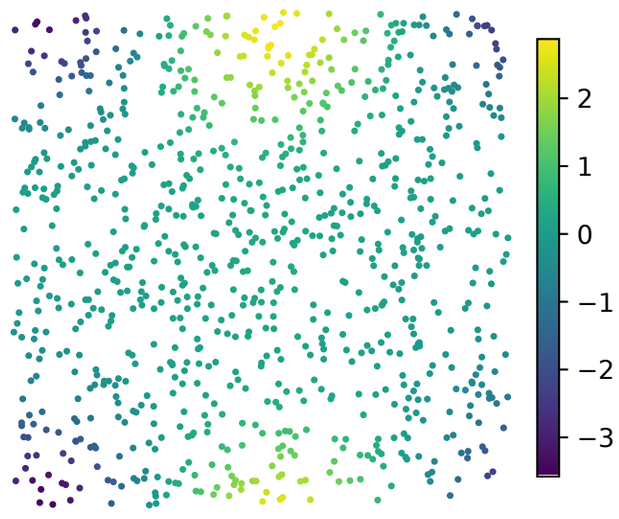
Replayed current 90k EMA checkpoint, saved source, and clean history. No target Y is supplied to the sampler.

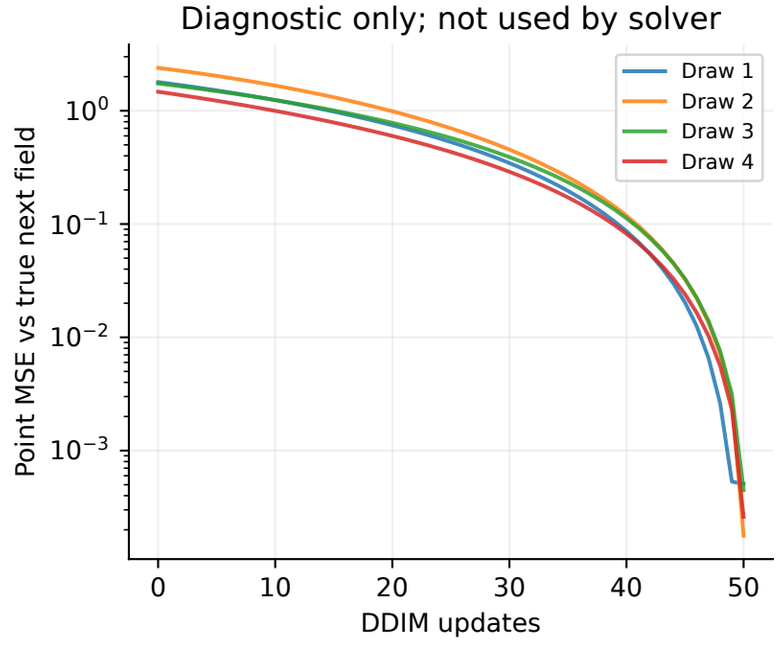










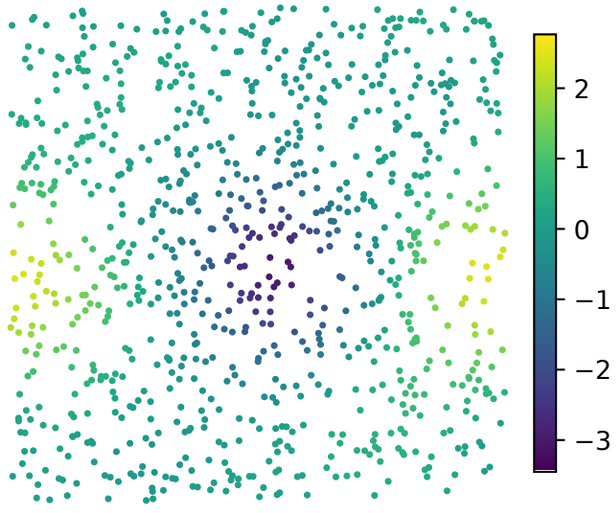


DDIM uses the predicted velocity to recover a clean estimate and a source estimate, then recombines them at the next level.
The 50-step replay matches the saved forecast within 4.77e-07; all eight history frames remain exact.

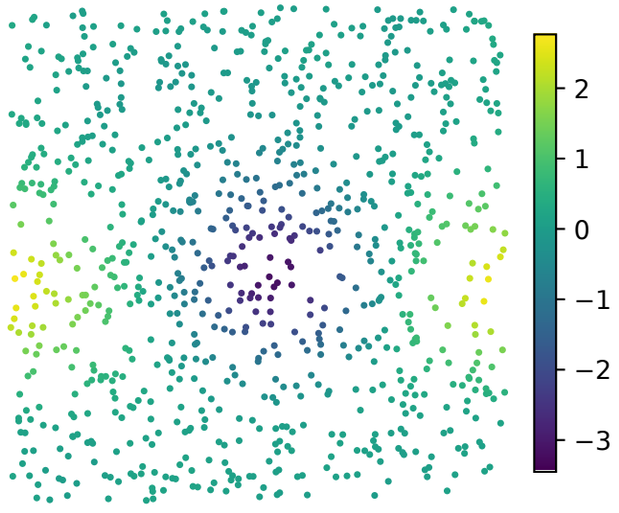
8. From random sources to predictive ensembles and rollout

Actual saved sixteen-step forecasts. Each member carries its own generated history and gets a fresh source at each physical step.

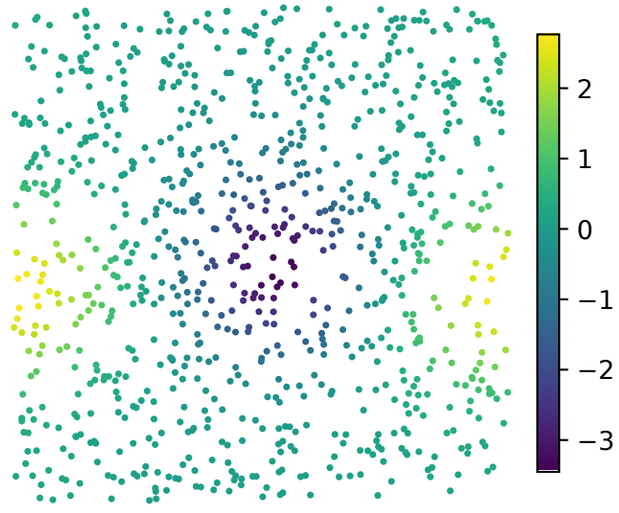
Physical lead 16 | member 1



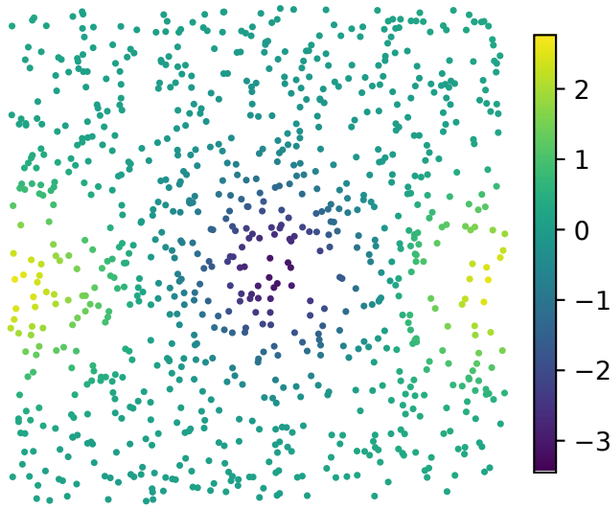
Physical lead 16 | member 2



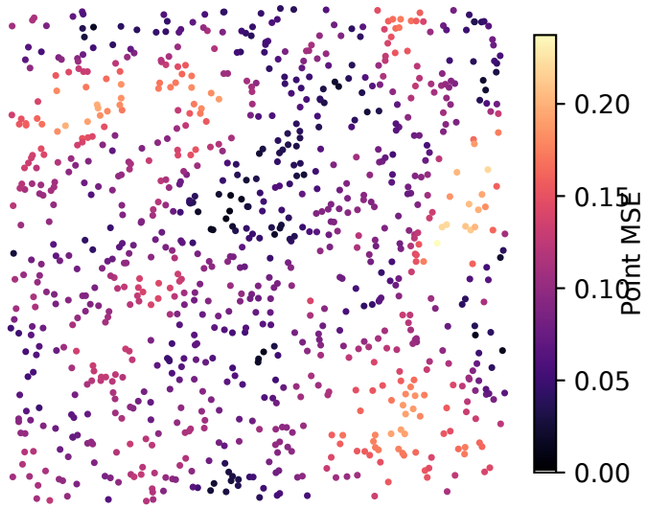
Physical lead 16 | truth



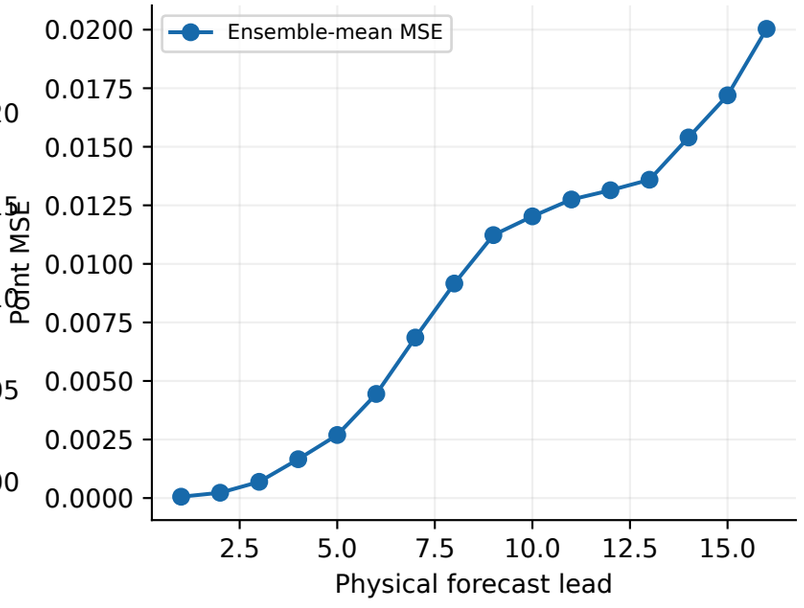
Four-member ensemble mean



Four-member spread (std, ddof=0)



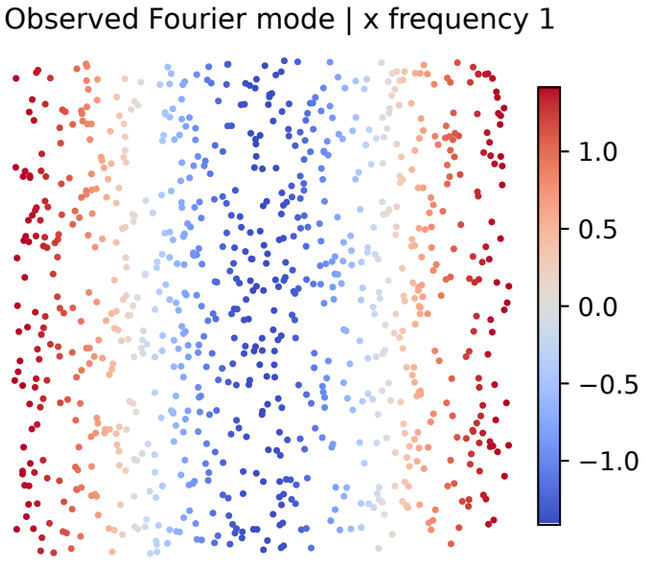
One validation example

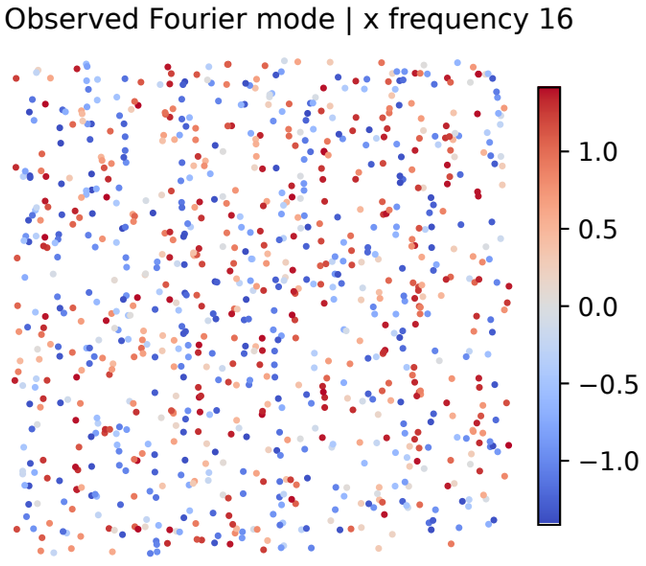


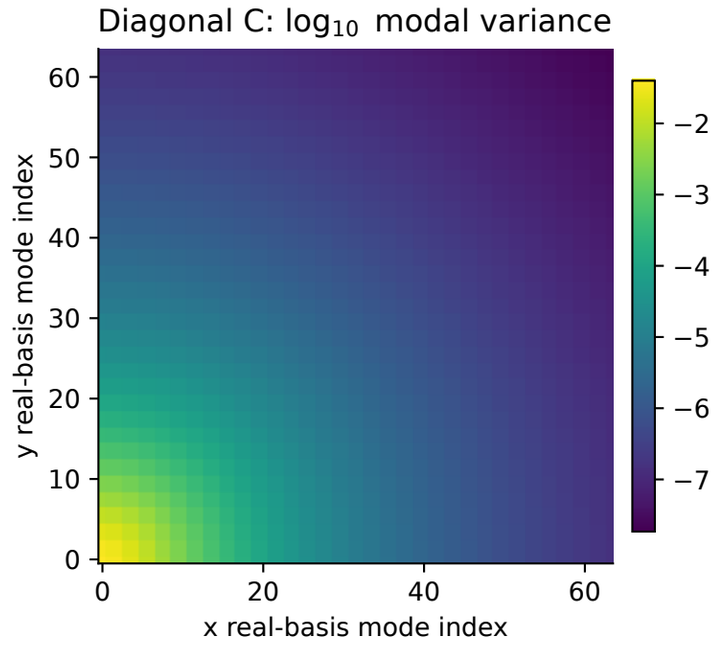
Randomness enters through the source; DDIM is deterministic conditional on source and history.
Four members illustrate ensemble behavior, but do not establish uncertainty calibration. This curve is one example, not the 32-case benchmark mean.

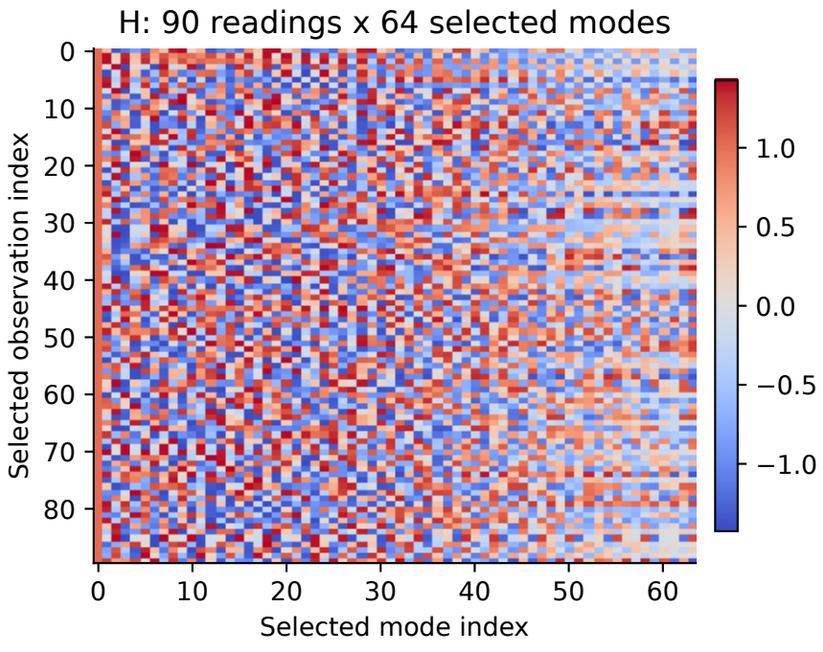
9. Where H, C, and G come from

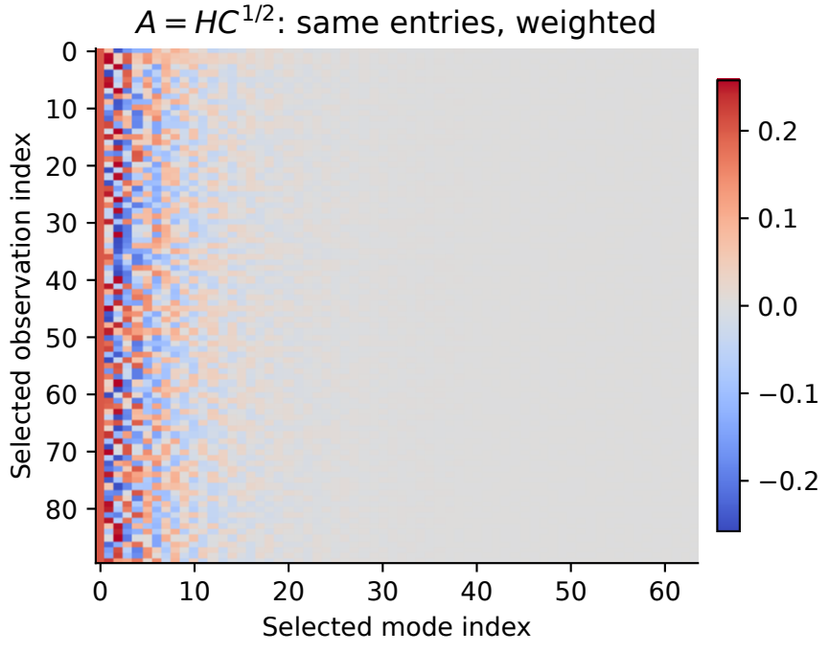
The finite Fourier basis has 4,096 modes. H applies the released RBF observation functionals to those modes.











$c_{k_x, k_y} \propto$
$\left(1 + (k_x^2 + k_y^2)/16\right)^{-3}$

$G = HCH^T = AA^T$

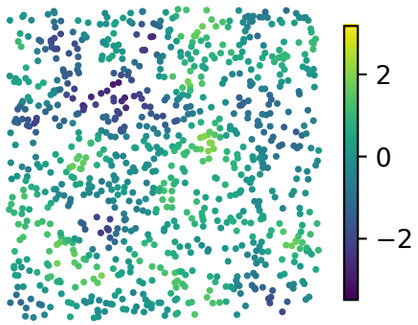
16-neighbor thin-plate RBF readouts
Degree-one polynomial; zero smoothing
All 4,096 prior modes retained
Only matrix slices are shown here.

The covariance is induced by both the finite prior and the observation functionals. H is fixed by geometry; it is not the learned backbone.
The real Fourier basis includes paired sine/cosine modes and its explicit Nyquist convention; matrix indices are not physical coordinates.

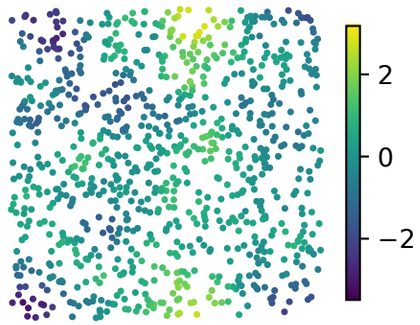
10. The observation path induces an RKHS field flow

Same linear lift at every level. Top: native readings. Middle: the lifted field at a fixed set of 900 new query points.

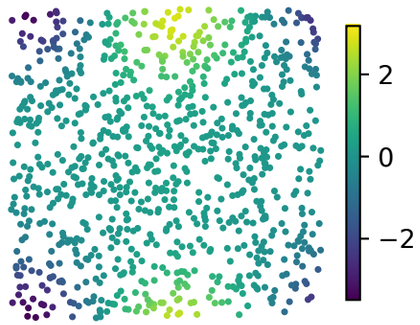
Observation state U | k=1000



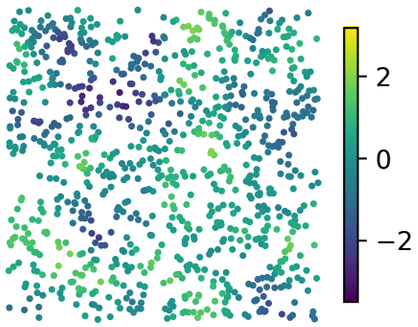
Observation state U | k=500



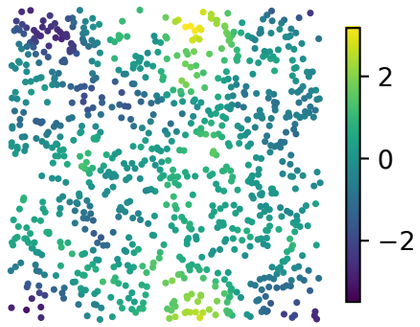
Observation state U | k=0



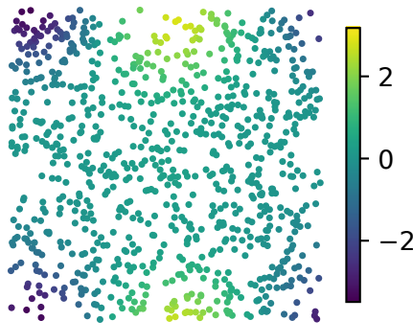
Lifted field f(U) | k=1000



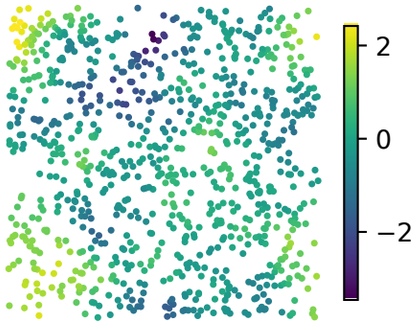
Lifted field f(U) | k=500



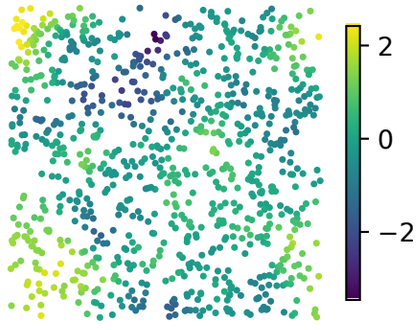
Lifted field f(U) | k=0



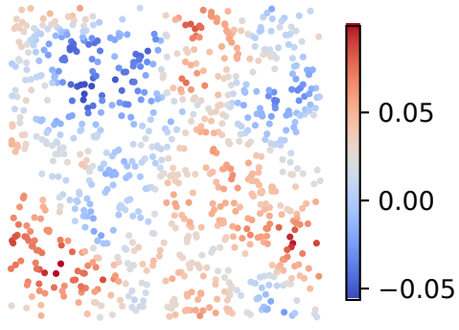
Target angular field velocity $S_X V$



Learned angular field velocity $S_X v_\theta$



Lifted velocity error | k=500



$\partial_\theta f_U = S_X V$: derivative and lift commute. Generation drift includes $\dot{\vartheta}$; angular velocity alone is not that drift.
Finite-difference derivative check: maximum error 7.03e-09. This is denoising evolution, not physical-time dynamics.

Executed smoke tests

All six requested/related cases passed: StocBench stochastic and deterministic, plus OceanSSH I-IV. Tests used the shared Python 3.10 / PyTorch 2.0.1 environment and an NVIDIA L40S, batch one, full configured spatial resolution, and fresh weights. No production training job was submitted. This is execution evidence, not trained model quality.

Real-data execution

Case	Resolution	Frames	Trainable parameters	Peak allocated GPU memory	Result
StocBench stochastic	64 x 64	2	167,131	0.150 GB	PASS
StocBench deterministic	64 x 64	2	167,173	0.150 GB	PASS
OceanSSH I	200 x 200	9	167,299	2.747 GB	PASS
OceanSSH II	200 x 200	9	167,299	2.747 GB	PASS
OceanSSH III	200 x 200	9	167,299	2.747 GB	PASS
OceanSSH IV	240 x 240	9	167,299	3.924 GB	PASS

Every case executed three distinct optimizer updates, one for each objective branch. A fourth computation repeats the second update after checkpoint reload to test replay. Each case strictly reloaded the final checkpoint and generated a finite fifty-step DDIM sample. Model, EMA, optimizer, and saved RNG replay differences were exactly zero in all six cases. The sample and checkpoint artifacts are included under `smoke/`.

StocBench checks preserved clean history exactly and replayed the same source exactly across all fifty steps. A second physical rollout step was finite. The deterministic case responded to current forcing. The stochastic model has no forcing input channel.

Ocean checks used observation-only inference data, verified conditioning dependence, and tested that changing values under zero availability masks does not change the network output. III additionally responded to SST. Empty observations produced finite samples. Fixed-source two-step replay was exact. Each case generated a fifty-step reconstruction and exported one correctly indexed NetCDF window; reload matched the exported values exactly. IV's 20-cell border was cropped, giving an inner 200-by-200 output. I-III used 200-by-200 patches. No full-period Ocean score was computed.

Geometry and backbone identity

The independent small-matrix tests compared transform-based precision solves against explicit dense inverses. Maximum absolute errors were 5.77e-15 (periodic) and 3.84e-13 (regional). Root/inverse-root reconstruction, the quadratic metric, and its gradient agreed with the dense reference within the stated 1e-10 tolerances.

The periodic 64-by-64 covariance agreed with the current Fourier endpoint kernel at selected native locations, including the Nyquist normalization. With nine frames, 30-by-30 geometry and identical weights, the generalized CNN matched the exact frozen current grid CNN with maximum absolute output difference 3.05e-16 (threshold 2e-6). This establishes backbone computation equivalence for that matched configuration, not equivalence of the new datasets or conditioning tasks.

Native StocBench integration

Both released callback pipelines completed using the new RKHS adapter's smoke EMA checkpoint. Each used one reference condition, four generated members, two DDIM steps and two physical rollout steps, with the extra long rollout disabled.

Task	Native callback artifacts	Result
Stochastic	Mean error, standard-deviation error, energy distance, enstrophy error	All four finite and saved
Deterministic	Mean error, standard-deviation error, rollout RMSE, enstrophy error	All four finite and saved

The completion receipts and artifact hashes are under `native_evaluation/stoc/` and `native_evaluation/det/`. These checks complement the fifty-step sampler checks; they do not imply that a two-step, four-member estimate is a publishable benchmark evaluation. The wrapper still depends on the external native StocBench checkout and local reference datasets.

Resolved implementation issue

The first attempt passed both StocBench cases, then stopped at Ocean I because PyTorch 2.0.1 disallows deterministic CUDA backward through its built-in replicated 3D padding. No numerical training failure was hidden. The adapter now pads regional spatial edges by explicit concatenation before the same convolution. An independent CPU reference using the built-in padding verified identical forward values for dilations 1, 2 and 4, with maximum gradient difference 1.78e-15. The complete six-case suite was then rerun successfully from fresh weights. The first-attempt log and final-attempt log are retained under `records/`.

Scientific limitations and next readiness checks

Ocean IV produced large finite pre-clipping gradient norms: approximately 4.12e6, 9.70e4 and 1.24e6 in its three distinct updates. Gradients were clipped at 1 and the model updated and sampled finitely, so this passes the execution check. It does not demonstrate healthy optimization. Its validation probe had about 99.68% finite targets; I-III had 100% in the selected probe. The current masked-residual precision quadratic can strongly emphasize sharp mask boundaries. That is a plausible contributor, not an isolated diagnosis. Review the missing-target likelihood/metric and scaling before a full IV training campaign.

The regional cosine covariance and reconstruction noise-pattern mixture are explicit new adaptations. A full Ocean campaign still needs a justified conditional-training/prior strategy and treatment of missing dense targets. No physics conservation law, calibrated posterior, deterministic StocBench ensemble collapse, convergence, or comparison to trained baselines was established here. No full OceanSSH training was resumed.

Receipts

`smoke/verification.json` gives all case checks and environment metadata. `smoke/geometry_verification.json` covers geometry and backbone parity. Each case has its own verification, data metadata, checkpoint and samples. `records/padding_verification.json` records the independent padding check. `records/code_origins.json` identifies reused and adapted files; `MANIFEST.sha256` records the delivered package contents.